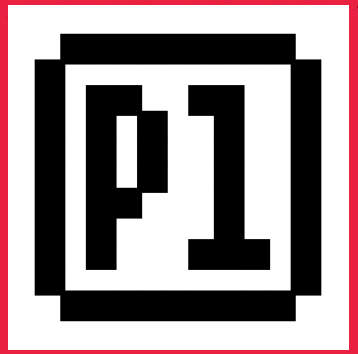
Matrixes



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

r33

19 / 2026

Primero, un repaso de

Arreglos

UNRN Universidad Nacional de Río Negro

```
tipo
nombre[TAMANO];
```

Declaración e inicialización

```
int arreglo[] = {0, 1, 2, 3, 4};
```



Declaración y acceso

```
int arreglo[MAX];
for (int i = 0; i < MAX; i++)
{
    arreglo[i] = i;
}
```




¿Preguntas?

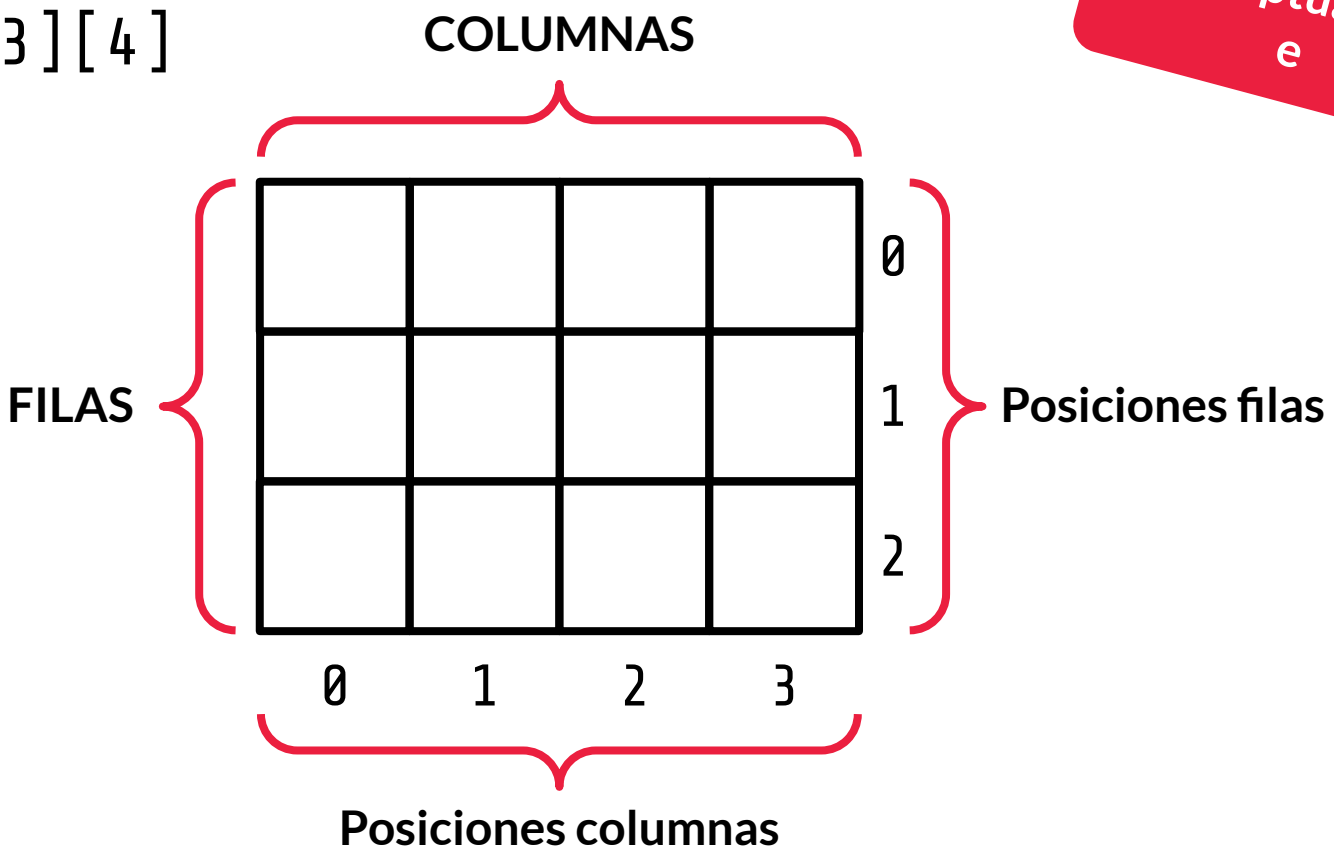
Matrices

UNRN Universidad Nacional de Río Negro

The Mat

```
filas x columnas
arreglo[3][4]
```

Conceptualmente



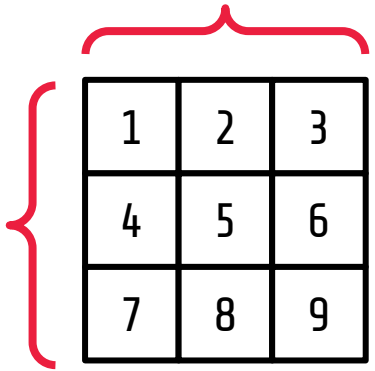
Definición estática

```
tipo identificador[FILAS][COLUMNAS];

int matrix[10][20];
```

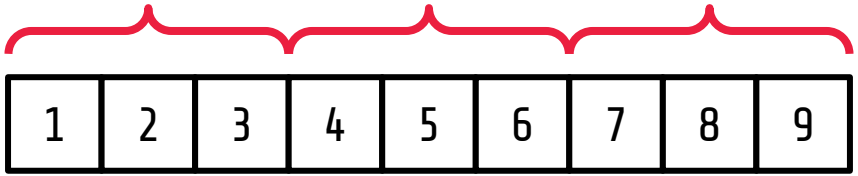
Definición e inicialización estática

```
int matriz[3][3] = {{1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9}};
```



En la memoria, la matriz es

```
int matriz[3][3] = {{1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9}};
```



¡Un bloque contiguo! En ‘row-major’ order

No *existen* las matrices en C

Carga de una matriz

```
int numeros[3][3];
int suma = 0;
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++)
    {
        numeros[i][j] = suma++;
    }
}
```

0	1	2
3	4	5
6	7	8

Impresión de una matriz

```
for (i = 0; i < 3; i++)
{
    printf("matriz=\n");
    for (j = 0; j < 3; j++)
    {
        printf("[%d]",numeros[i][j]);
    }
    printf("\n");
}
```

```
matriz=
[0][1][2]
[3][4][5]
[6][7][8]
```

0	1	2
3	4	5
6	7	8

Funcionan igual que los arreglos

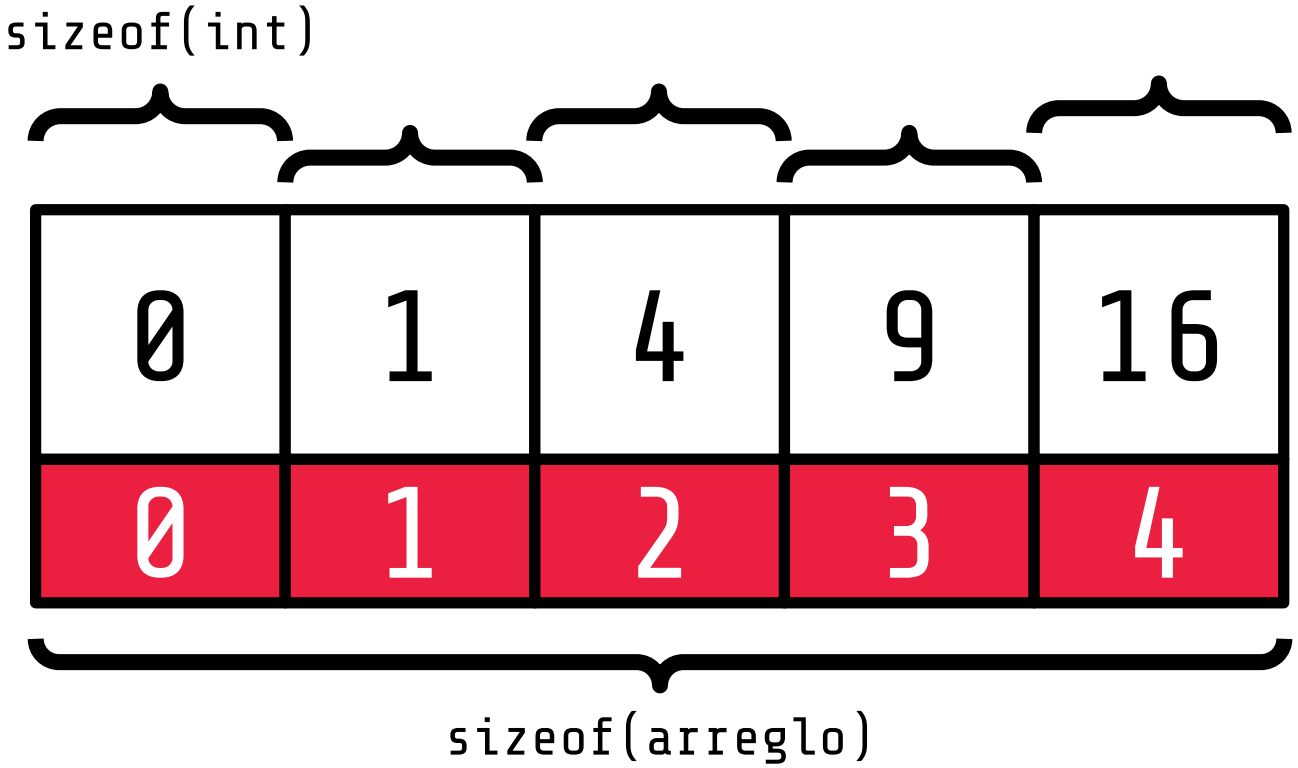
Los arreglos *estáticos* solo pueden ser creados de un tamaño fijo al compilar


¿Preguntas?

Matrices y funciones

UNRN Universidad Nacional de Río Negro

Para este `int arreglo[]` de `size_t` largo



`largo` es cuantos `int` tiene el arreglo

¿Cuánto daba esto?

```
size_t largo(int arreglo[])
{
    return sizeof(arreglo) / sizeof(arreglo[0]);
}
```

¿Cuánto daba esto?

```
size_t largo(int arreglo[])
{
    return sizeof(arreglo) / sizeof(arreglo[0]);
}
```

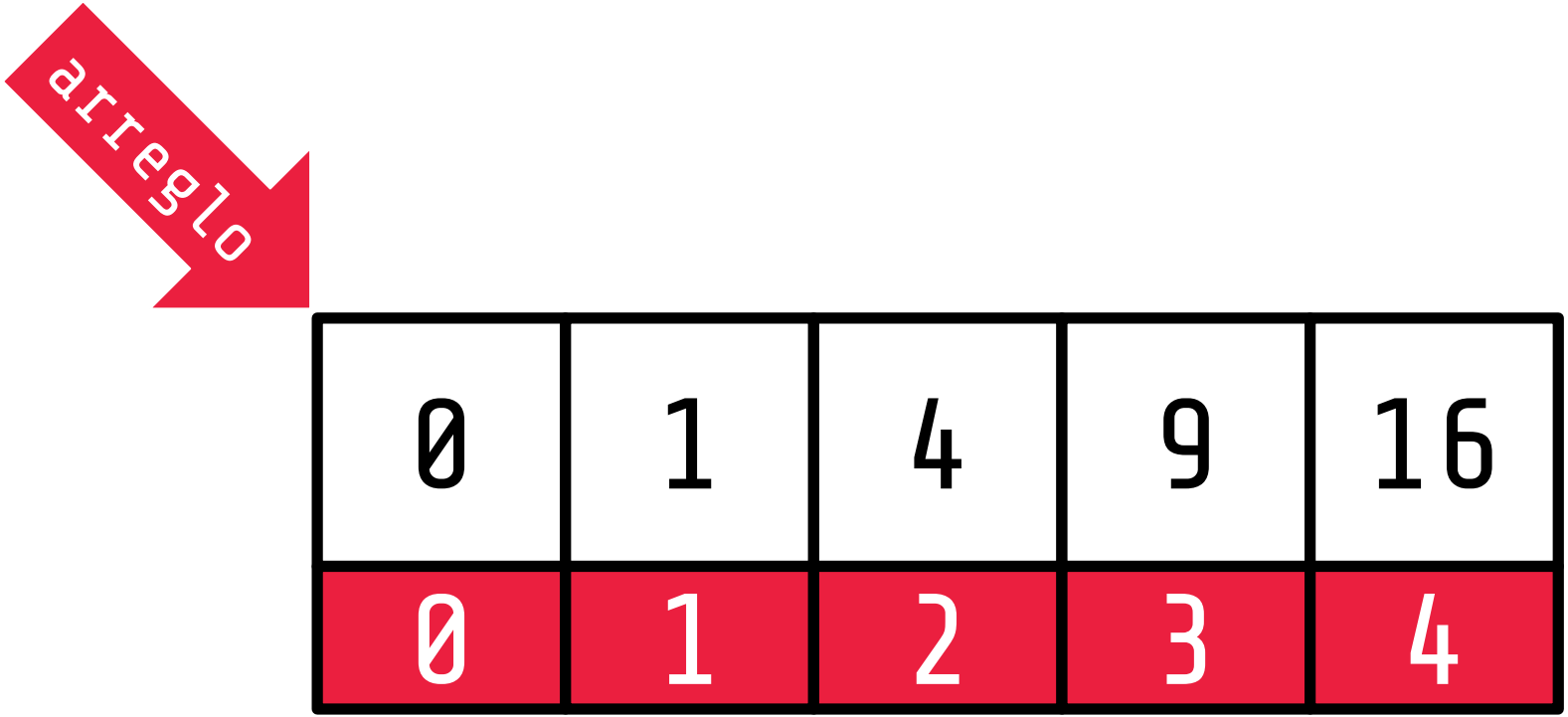
¡8!

¿Cuánto daba esto?

```
size_t largo(int arreglo[])
{
    return sizeof(arreglo) / sizeof(arreglo[0]);
}
```

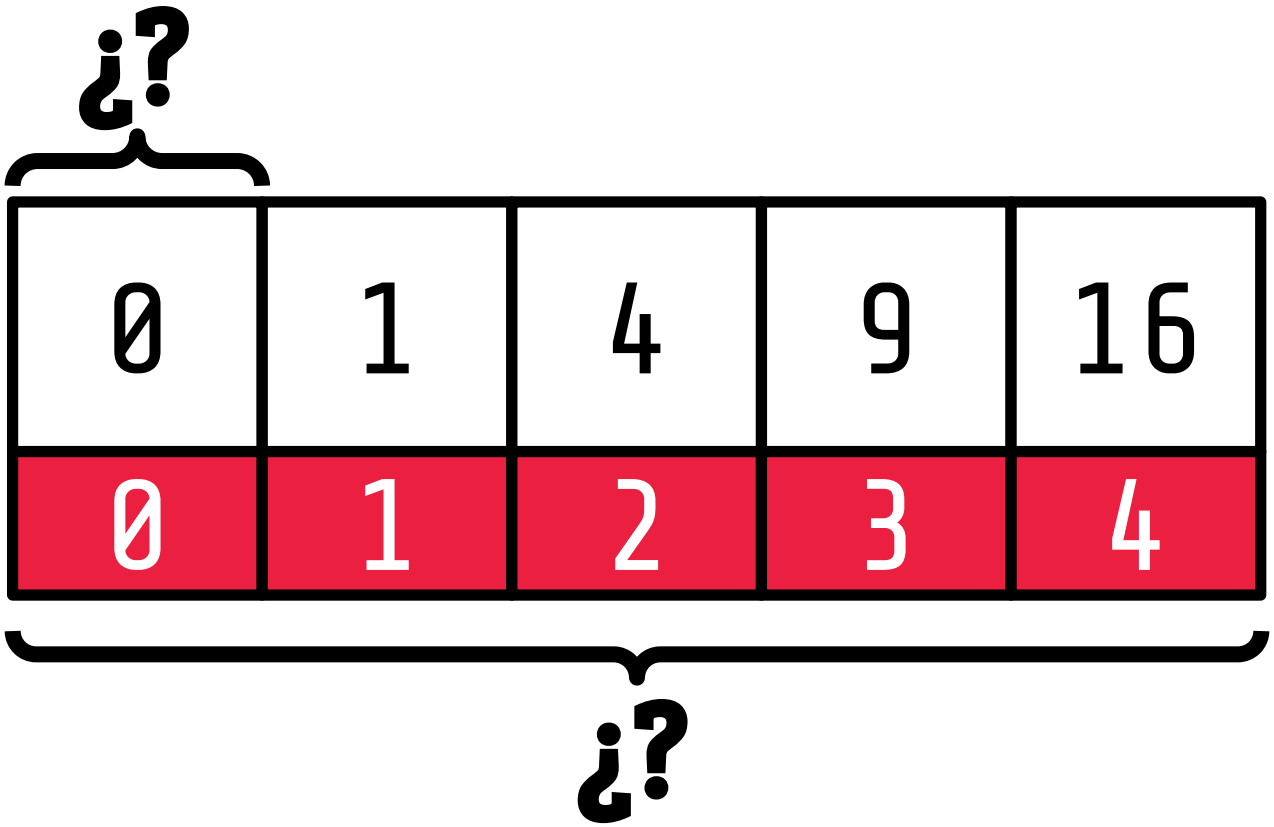


Para esta `función(int arreglo[])`



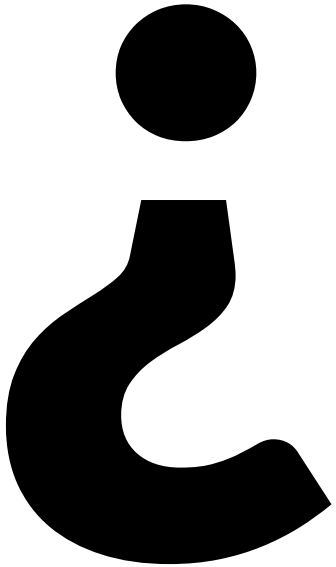
`arreglo` es la dirección del primer elemento

En esta `función(int arreglo[], size_t largo)`



¿Cuánto valen dentro de la función?


¿Preguntas?



y con una matriz



¿Cuánto da esto?

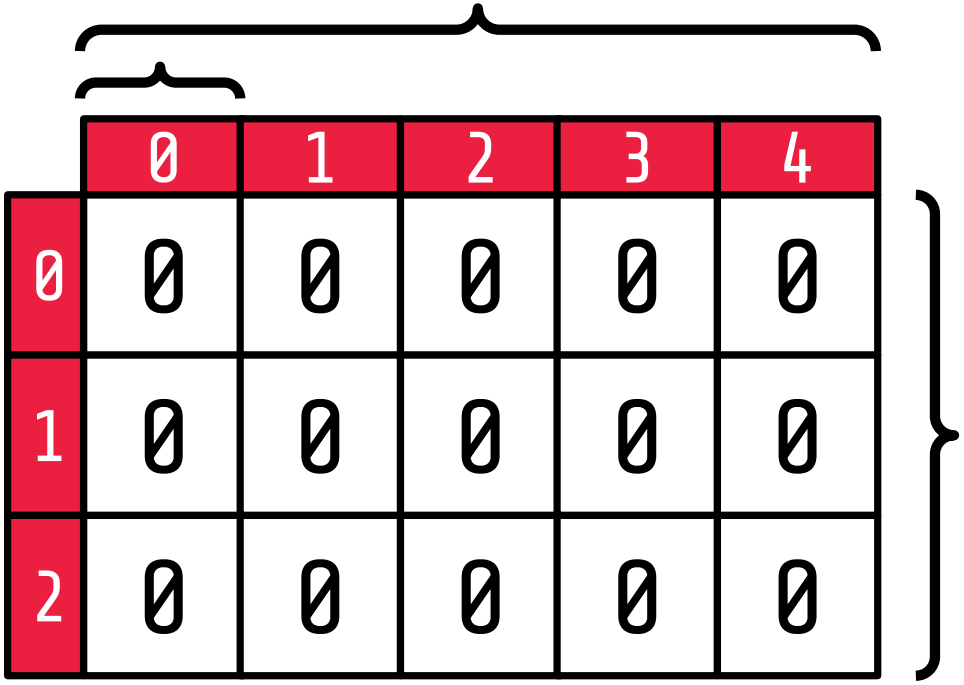
```
int matrix[100][100];
tamaño(matrix);
.....
size_t funcion(int matriz[][])
{
    return sizeof(matriz);
}
```

¿Cuánto da esto?

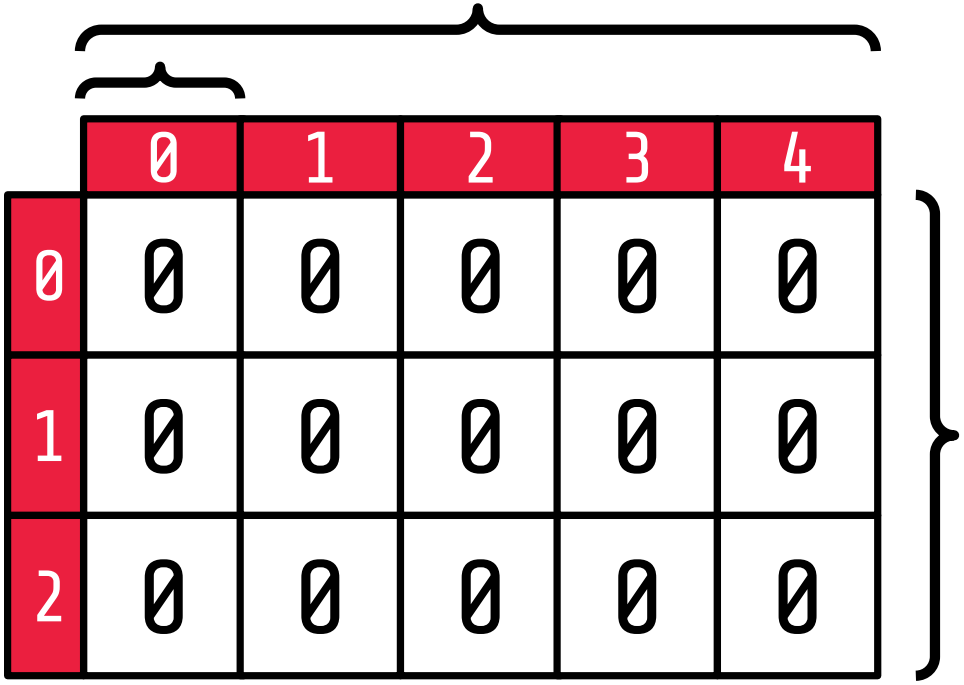
```
int matrix[100][100];
tamaño(matrix);
.....
size_t funcion(int matriz[][])
{
    return sizeof(matriz);
}
```

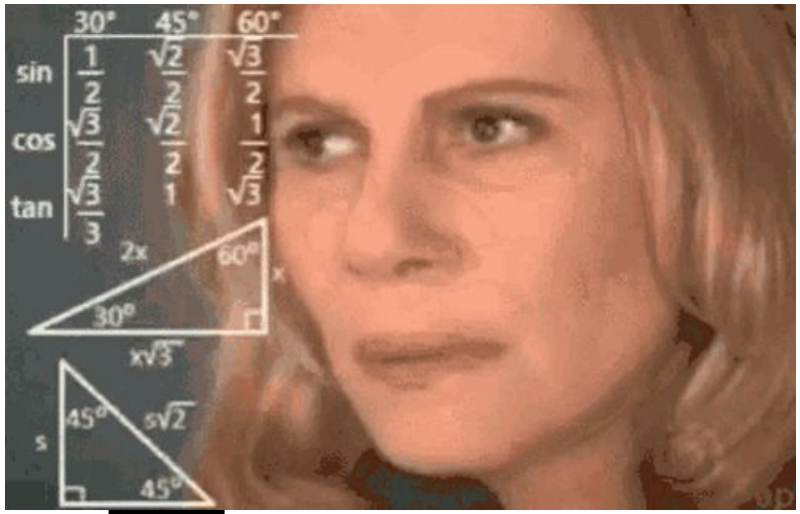
¡No sabe cual es el ancho de cada fila!

¿Y en `funcion(int matrix[][])`?

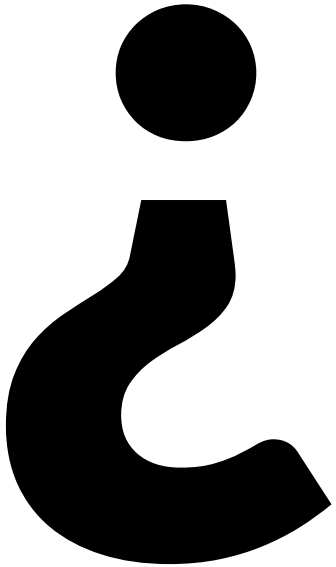


Cuánto mide cada parte para `int matrix[][]`





También decae a dirección de memoria...



Como hacemos



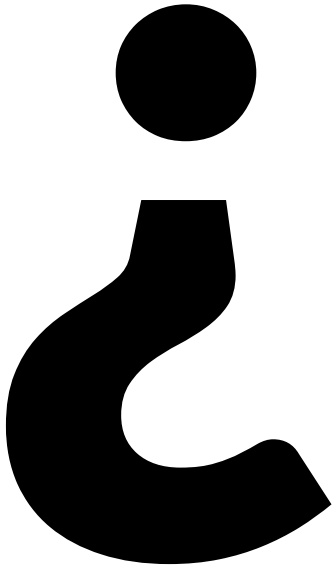
Una función que recibe una matriz

```
#define COLUMNAS 3

void imprime_matriz(size_t filas,
                    size_t columnas,
                    int matrix[][COLUMNAS]);
```

`matrix` decae a un puntero al primer elemento

[un arreglo de `COLS` enteros (`int (*)[COLS]`).]



Por qué



{ **Como era una matriz en memoria...** }

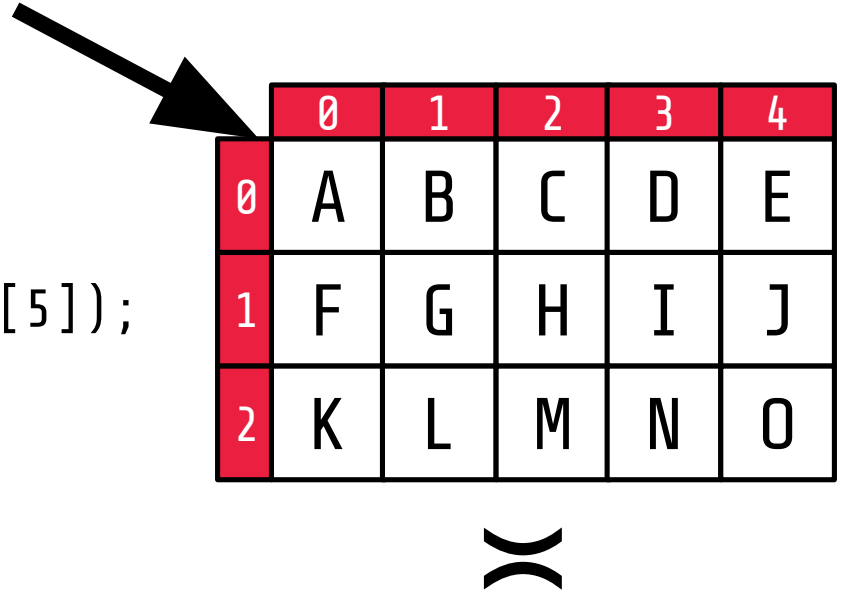
```
char matrix[3][5]);
```

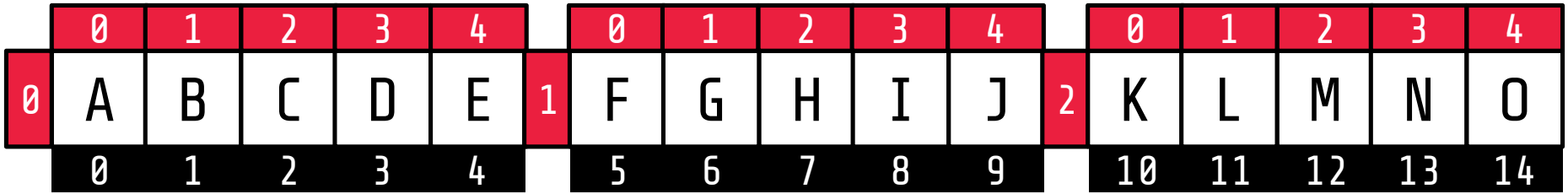
	0	1	2	3	4
0	A	B	C	D	E
1	F	G	H	I	J
2	K	L	M	N	O



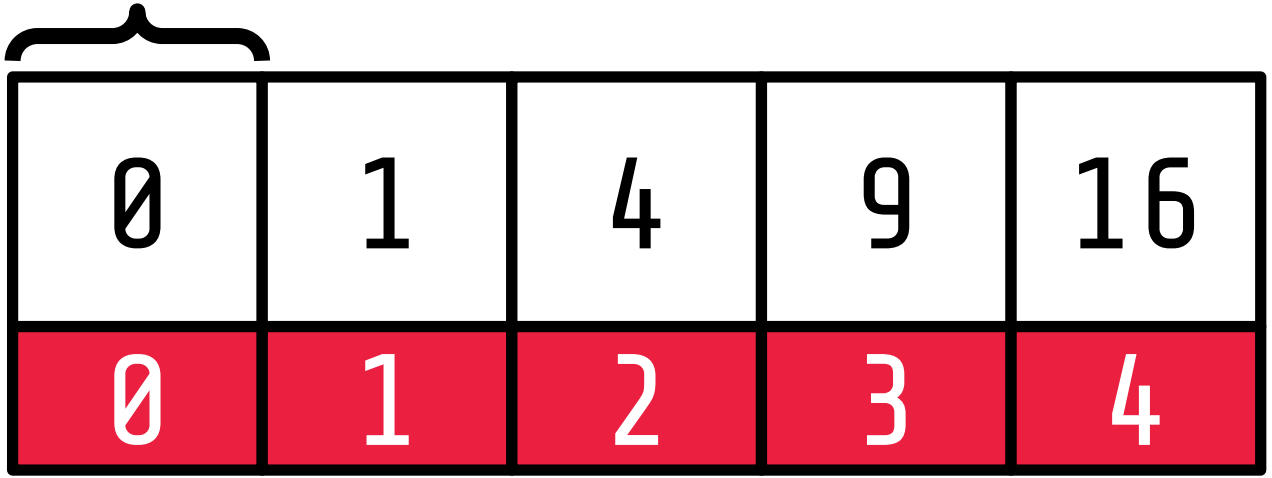
	0	1	2	3	4		0	1	2	3	4		0	1	2	3	4
0	A	B	C	D	E	1	F	G	H	I	J	2	K	L	M	N	O
	0	1	2	3	4		5	6	7	8	9		10	11	12	13	14

```
char matrix[3][5]);
```





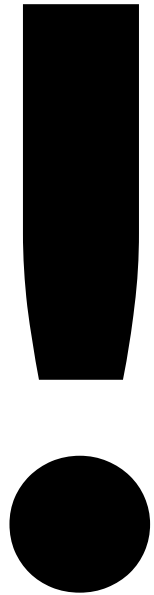
En esta `función(int arreglo[], size_t largo)`



En un `arreglo`, ¿que hacen los `[]`?



```
arreglo[i]
≍
arreglo + i
```



```
char matrix[3][5]);
```

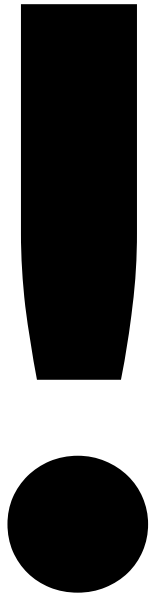
	0	1	2	3	4
0	A	B	C	D	E
1	F	G	H	I	J
2	K	L	M	N	O



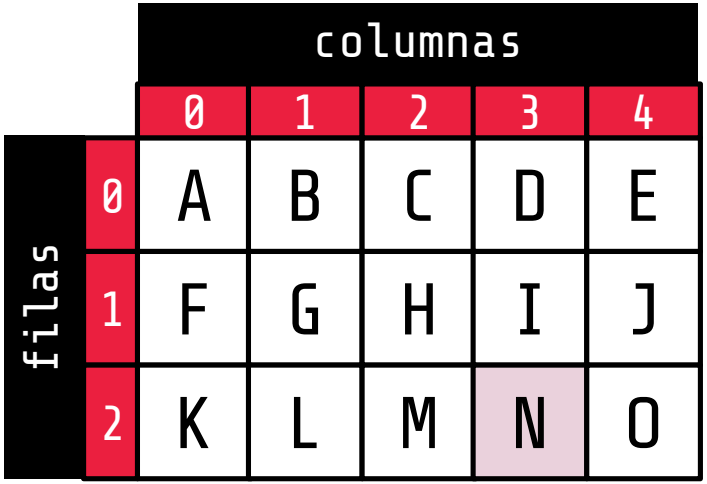
	0	1	2	3	4		0	1	2	3	4		0	1	2	3	4
0	A	B	C	D	E	1	F	G	H	I	J	2	K	L	M	N	O
	0	1	2	3	4		5	6	7	8	9		10	11	12	13	14



```
matriz[i][j]
≍
matrix+((i*COLUMNAS)+j
```

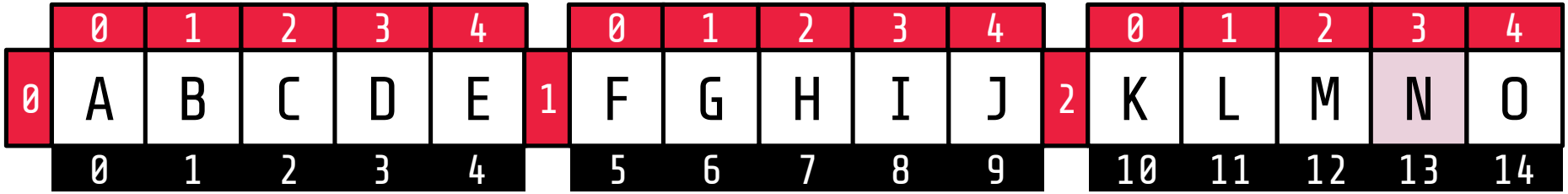


```
char matrix[3][5];
≍
(i*COLUMNAS)+j
```




```
matrix[2][3] ≍
(i*5) + 3 =
(2*5)+3 =
10+3=13
```

≍




¿Preguntas?

Ya vamos a ver cómo liberarnos de esta restricción

```
#define COLUMNAS 3
#define FILAS 3

void muestra(size_t filas,
             size_t columnas,
             int matrix[FILAS][COLUMNAS]);
```

Con la sintaxis ALV/VLA

```
void muestra(size_t filas,
             size_t columnas,
             int matrix[filas][columnas]);
```

Con `filas` / `columnas` declaradas antes que `matrix`


¿Preguntas?

Matrices dinámicas

UNRN Universidad Nacional de Río Negro

Primer enfoque

```
int** matriz = (int**)malloc(filas * sizeof(int*));

for(int i = 0; i < filas; i++){
    matriz[i] = (int *) malloc(columnas * sizeof(int));
}
```

¿Cuáles son los pasos acá?

Matrices dentadas (jagged)

```
int** matriz = (int**)malloc(filas * sizeof(int*));

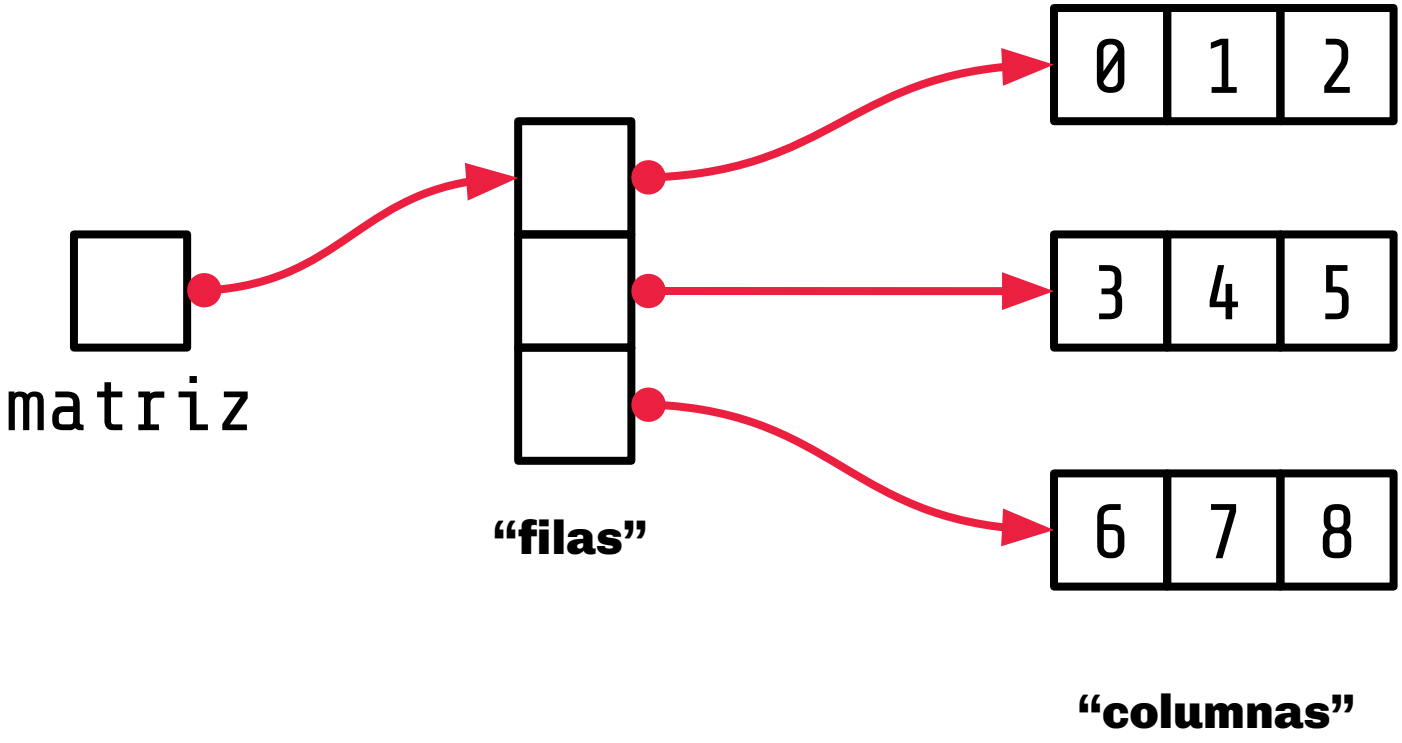
for(int i = 0; i < filas; i++){
    matriz[i] = (int *) malloc(columnas * sizeof(int));
}
```

¿Son lo mismo?

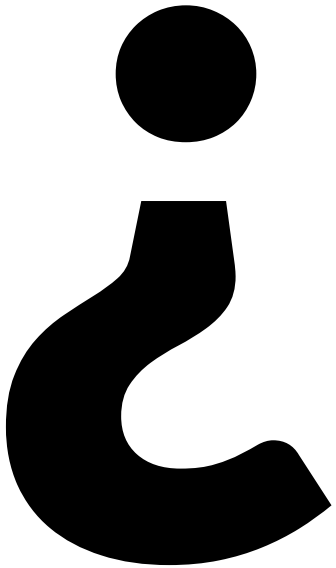

int
int*
int**
int***

{ El uso de la matriz es igual después }

A efectos prácticos, este es el resultado



una matriz dentada *técnicamente* ocupa más espacio



Cuantos `free` son necesarios

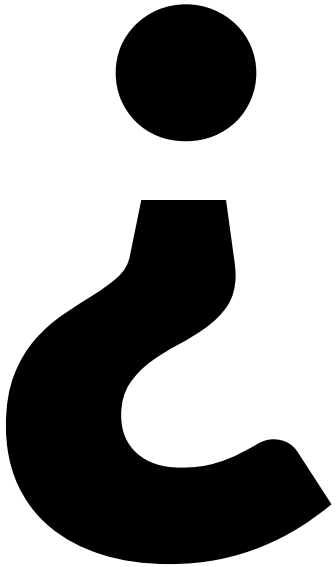


La liberación de memoria es paso a paso

```
for(int i = 0; i < filas; i++){
    free(matriz[i]);
}
free(matriz);
```

Muy importante que sea de ‘dentro a fuera’


¿Preguntas?

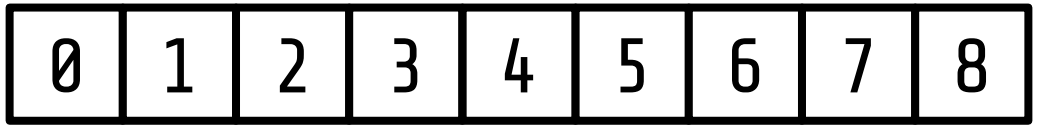


No se puede crear una matriz en un bloque contiguo



Dinámica como un único bloque

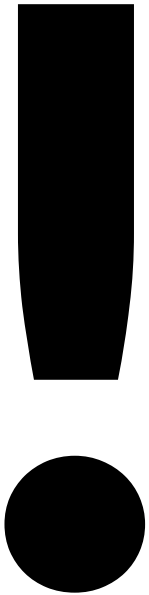
```
int* matriz = (int*) malloc(filas * columnas * sizeof(int));
```



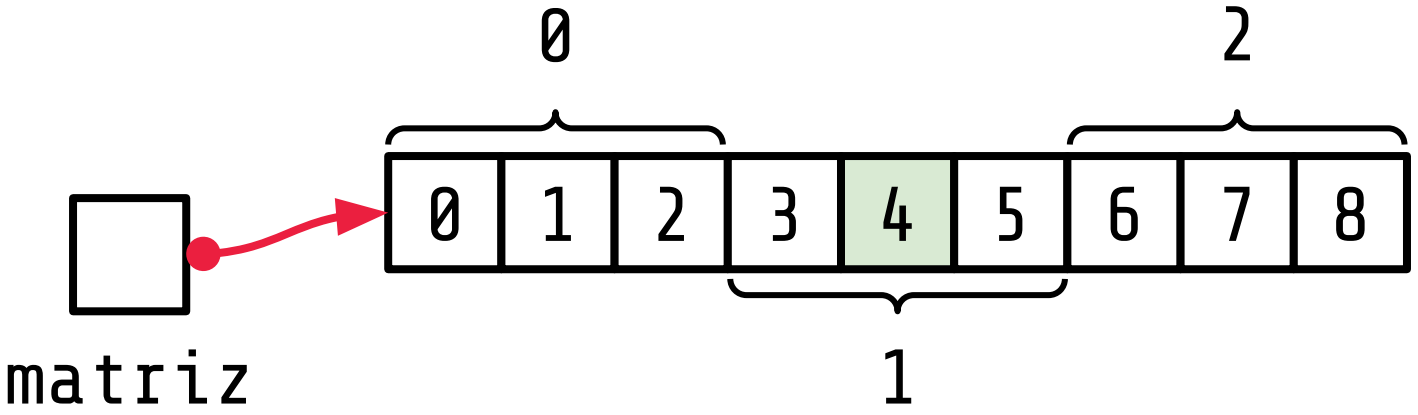

¿ Y los [ ] [ ] ?



No están



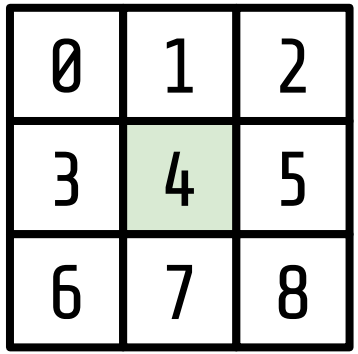
En memoria, queda algo como

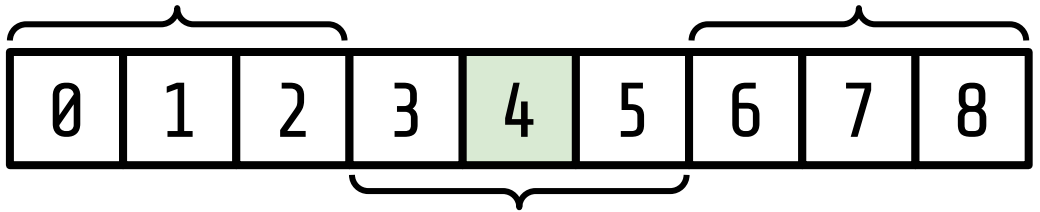


Pero no tenemos []

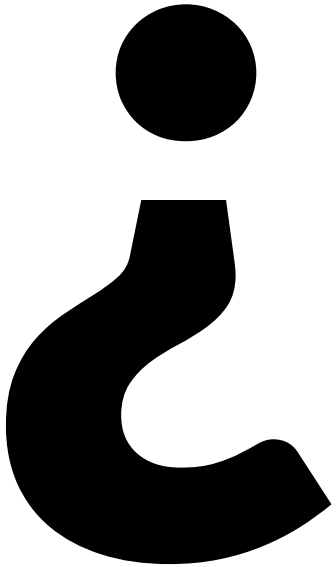
Es necesario simularlos

```
int valor = matriz[(i * columnas) + j];
```






¿Preguntas?



No se puede usar un cast

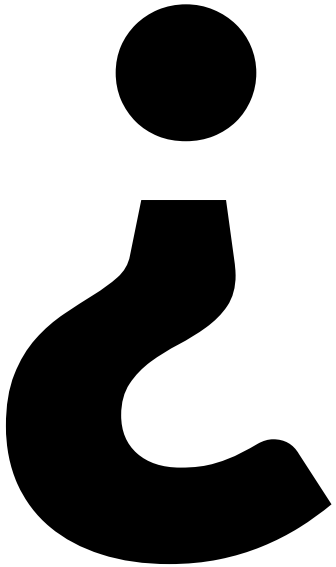


¿Cual es el resultado?

```
int a[3][4];
int **p;
p = a;
```

`ptr` es un puntero a un array de 4 enteros

```
int a[3][4];
int (*ptr)[4] = (int (*)[4])a;
```



Y esto cómo se usa

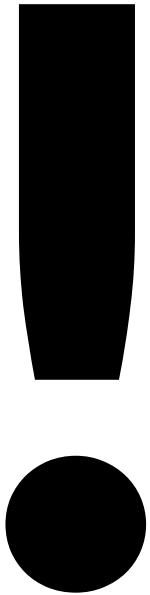


Cast de arreglo a arreglo de filas de columna ancho

```
int (*matriz)[columnas] = malloc(
                          sizeof(int) * columnas * filas);
```



Y transformamos en matriz el arreglo



Como TDA

UNRN Universidad Nacional de Río Negro

Una opción más simple

Definimos la estructura

```
typedef struct {
    size_t filas;
    size_t columnas;
    double *datos;
} matriz_t;
```



Y las operaciones

```
matriz_t* matriz_crear(size_t filas, size_t columnas)

double matriz_get(const matriz_t *m, size_t i, size_t j)

void matriz_put(const matriz_t *m, size_t i, size_t j, double val)

void matriz_destruir(matriz_t **m)
```

Y encapsulamos la complejidad del acceso y modificación

Invariantes

1. el puntero de la matriz_t no puede ser NULL
2. los datos no pueden ser NULL
3. filas y columnas no pueden ser 0
4. filas no puede ser > que SIZE_MAX / columnas
   (no podemos crear una matriz SIZE_MAX x SIZE_MAX)


¿Preguntas?

Constructs



UNRN Universidad Nacional de Río Negro

¡Para armar juegos!

```
typedef struct {
    char entidad;
    char terreno;
    bool transitado;
} celda_t;

celda_t tablero[10][10];
mapa[0][0].terreno = 'A';

// O dinámica
celda_t (*tablero)[cols] = malloc(f * c * sizeof (celda_t));
tablero[i][j].transitado = true;
```



Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

unrn.edu.ar

UNRN Universidad Nacional de Río Negro

| unrionegro